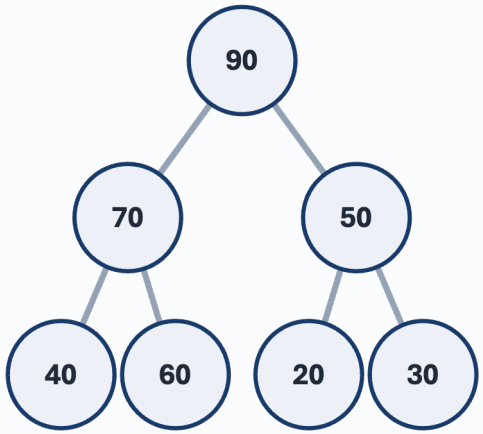
Heap Data Structure
Heap Sort Algorithm

- Definition and structure of the heap
- Core ADT operations and implementation logic
- How heap sort works

NGAN Chi Ho
11552520
COMP8090SEF Data Structures and Algorithms
Task 2



Max heap
[90, 70, 50, 40, 60, 20, 30]

Study Aim and Scope

Heap Key ideas

- Special complete binary tree
- Supports priority queues
- Efficient array representation
- Comparison-based sorting algorithm
- Built directly on heap properties
- Guaranteed O(n log n) time

Why heap matters

- Efficient access to the maximum or minimum element.
- Standard implementation of a priority queue.
- Useful in scheduling and shortest-path algorithms.

Link to heap sort

• Heap sort first builds a max heap.
• The root gives the largest unsorted value.
• Repeated restoration of the heap yields O(n log n) performance.

Heap: Definition and Structural Properties

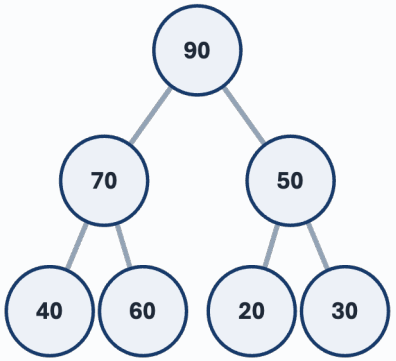
Core properties

- A heap is a complete binary tree.
- Max Heap: Parent ≥ Children
- Min Heap: Parent ≤ Children
- Only local parent–child order is guaranteed.

Array representation

left child = 2i + 1
right child = 2i + 2
parent = (i - 1) // 2

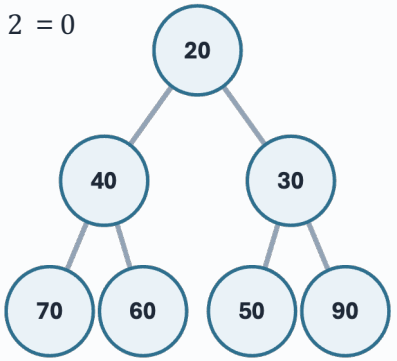
Max heap example



[90, 70, 50, 40, 60, 20, 30]

Min heap example

((1) - 1) // 2 = 0



2(1) + 1 = 3

2(1) + 2 =4

[0, 1, 2, 3, 4, 5, 6]
[20, 40, 30, 70, 60, 50, 90]

Real-World Applications of Heaps

- **Emergency Room Triage**
- Patients are prioritized by **urgency**, not arrival time.
- Max-Heap ensures the most critical cases are handled first.

- **System**
- If your system frequently needs the **highest or lowest priority** item, Heaps are the most efficient solution.

- Widely used across software engineering to optimize performance in data-heavy environments.

- **Top 10 Song**
- Used to identify top-ranked songs or products in massive datasets.
- Avoids the cost of a full system sort, saving significant computational power.

- **Graph Algorithms (Dijkstra's)**
- Uses a **Min-Heap** to efficiently select the next closest node.
- Essential for GPS navigation and network routing.

Heap Sort: Procedure and Example

1. Build a max heap
Largest value moves to the root.

2. Swap root with last
The largest value is fixed at the end.

3. Reduce heap size
The sorted suffix is excluded.

4. Heapify root
Heap property is restored.

Worked example

[20, 40, 30, 70, 60, 50]

	Index	0	1	2	3	4	5
Original array	Value	20	40	30	70	60	50
After build max heap	Value	70	60	50	40	20	30
After 1st swap	Value	30	60	50	40	20	[70]
After heapify	Value	60	40	50	30	20	[70]
After 2nd swap	Value	20	40	50	30	[60]	[70]
After heapify	Value	50	40	20	30	[60]	[70]
After 3rd swap	Value	30	40	20	[50]	[60]	[70]
After heapify	Value	40	30	20	[50]	[60]	[70]
After 4th swap	Value	20	30	[40]	[50]	[60]	[70]
After heapify	Value	30	20	[40]	[50]	[60]	[70]
After 5th swap	Value	20	[30]	[40]	[50]	[60]	[70]

Why the algorithm works

- The root of a max heap always stores the current maximum.
- After a swap, that maximum is placed in its final sorted position.
- Heapify restores the structure on the remaining unsorted prefix.
- Repeating the process produces ascending order from right to left.

Implementation Logic in Python

Key functions from the report: heapify() and heap_sort()

```
def heapify(arr, n, i):
    largest = i
    left = 2*i + 1
    right = 2*i + 2

    if left < n and arr[left] > arr[largest]:
        largest = left
    if right < n and arr[right] > arr[largest]:
        largest = right
    if largest != i:
        arr[i], arr[largest] = arr[largest], arr[i]
        heapify(arr, n, largest)

def heap_sort(arr):
    n = len(arr)
    for i in range(n//2 - 1, -1, -1):
        heapify(arr, n, i)
    for i in range(n-1, 0, -1):
        arr[i], arr[0] = arr[0], arr[i]
        heapify(arr, i, 0)
```

How to read the code

- heapify() checks the root against its children and pushes smaller values downward.
- The first loop builds a max heap from the original array.
- The second loop swaps the root with the last unsorted element.
- After each swap, heapify() restores order in the reduced heap.

Implementation insight

The algorithm is efficient because child indices are computed directly from array positions.

Heap ADT: Principal Operations

Operation	Purpose	Time Complexity
peek()	Returns the root element without removing it.	O(1)
insert(x)	Places a new element at the end, then restores the heap by sift-up.	O(log n)
extractMax() / extractMin()	Removes the root, moves the last element to the top, then sift-down restores order.	O(log n)
heapify()	Restores the heap property for a subtree after a violation.	O(log n)
buildHeap()	Builds a heap bottom-up from an unsorted array.	O(n)

Key interpretation: most heap operations are efficient because the height of a complete binary tree grows approximately as log n.

Applications, Strengths, and Limitations

Applications

- Priority queues
- Task scheduling
- Dijkstra and algorithms
- Efficient selection of max or min values
- Constant-time access to the maximum or minimum element

Strengths

- Fast access to the root element
- Compact array-based storage
- Guaranteed O(n log n) sorting time
- In-place sorting with low extra memory

Limitations

- Not fully sorted internally
- Searching for an arbitrary item is inefficient
- Heap sort is not stable
- Often less cache-friendly than quicksort

Conclusion

Summary

- The heap is a complete binary tree with a parent–child priority rule.
- Heap sort uses the heap property to place the largest remaining value into final position at each pass.
- The algorithm achieves O(n log n) time in all cases and uses very little extra memory.
- Heap and heap sort show how a suitable data structure enables an efficient algorithm.
- Heap is an efficient and practical data structure for managing prioritized elements.

Thank you.